

1. Linear Regression

Goal

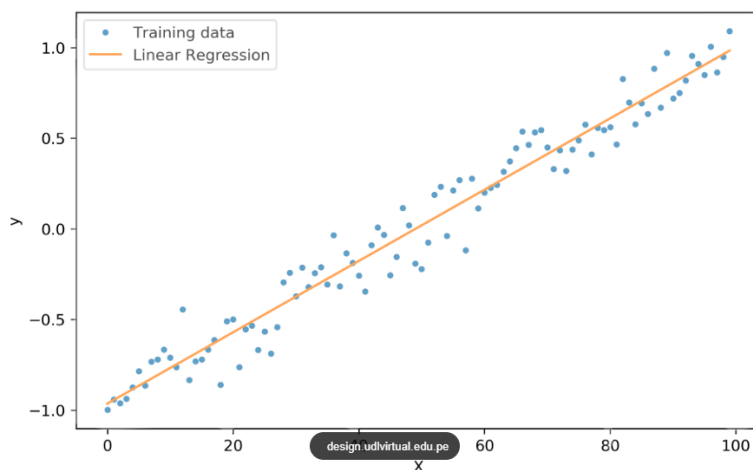
Predict a **continuous number**.

Examples:

- House price
- Salary

Idea

Draw the **best-fit line** through the data.



Equation:

$$y = mx + c$$

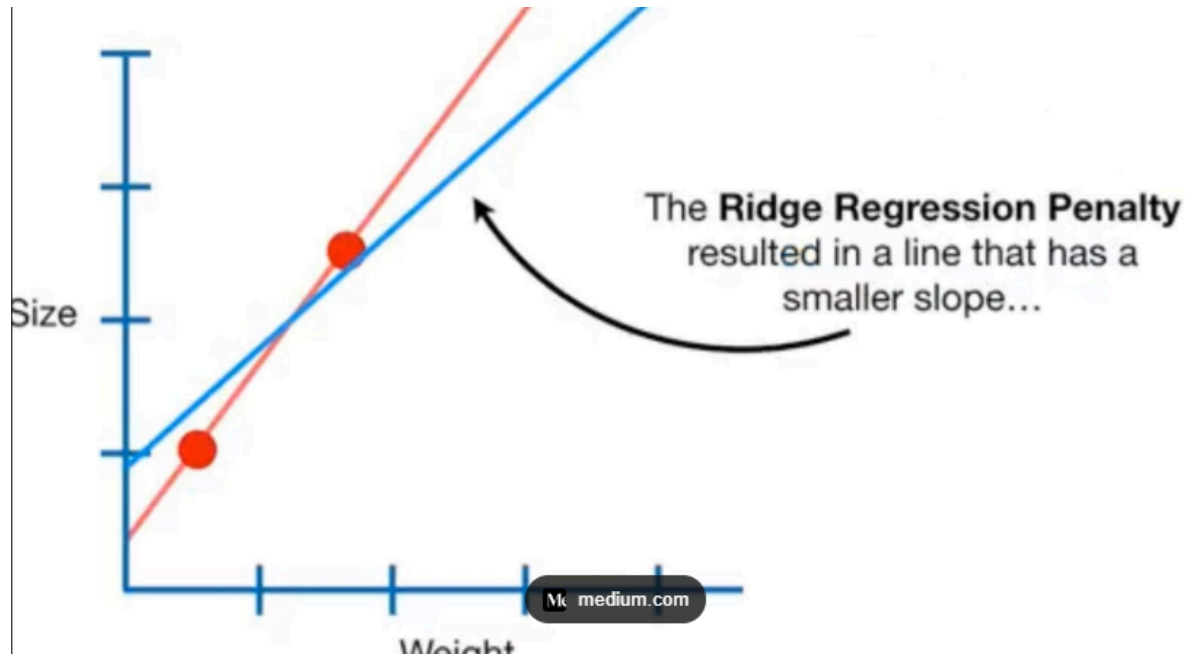
- $x \rightarrow$ input
- $y \rightarrow$ prediction
- $m \rightarrow$ slope
- $c \rightarrow$ intercept

Python

```
from sklearn.linear_model import LinearRegression
model = LinearRegression()
model.fit(X_train, y_train)
y_pred = model.predict(X_test)
```

2 .Ridge Regression

Ridge Regression adds a penalty term to the Linear Regression cost function to prevent overfitting.



Equation:

Ridge Regression Loss (L2 Regularization)

Ridge adds a penalty equal to the **sum of the squares of the coefficients**:

$$\text{Loss} = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2 + \lambda \sum_{j=1}^p w_j^2$$

Where:

- y_i = Actual value
- $\hat{y}_i$ = Predicted value
- w_j = Weight (coefficient)
- λ (called **alpha** in scikit-learn) = Regularization strength

Python

```
from sklearn.linear_model import Ridge
```

```
model = Ridge(alpha=1.0)
```

```
model.fit(X_train, y_train)
```

```
y_pred = model.predict(X_test)
```

3. Lasso Regression

Lasso Regression Loss (L1 Regularization)

Lasso adds a penalty equal to the **sum of the absolute values of the coefficients**:

$$\text{Loss} = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2 + \lambda \sum_{j=1}^p |w_j|$$

Where:

- y_i = Actual value
- $\hat{y}_i$ = Predicted value
- w_j = Weight (coefficient)
- λ (called **alpha** in scikit-learn) = Regularization strength

Python

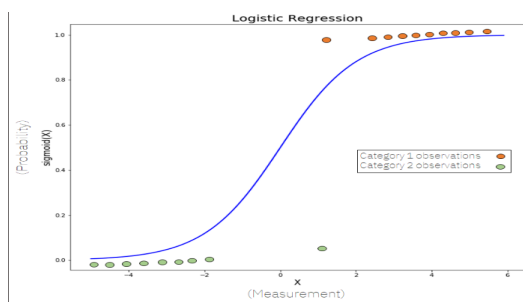
```
from sklearn.linear_model import Lasso
model = Lasso(alpha=1.0)
model.fit(X_train, y_train)
y_pred = model.predict(X_test)
```

2. Logistic Regression(Classification)

So logistic regression first draws a line $z = wx + b$. Then passes it through the Sigmoid Function.

$$z = wx + b$$

$$P(y = 1 | x) = \sigma(z) = \frac{1}{1 + e^{-z}}$$



```
from sklearn.linear_model import LogisticRegression
model = LogisticRegression()
model.fit(X_train, y_train)
y_pred = model.predict(X_test)
```

3. K-Nearest Neighbors (KNN)

1. Euclidean Distance (Most Common)

The main KNN formula is:

$$d = \sqrt{\sum_{i=1}^n (x_i - y_i)^2}$$

or for two features:

$$d = \sqrt{(x_1 - y_1)^2 + (x_2 - y_2)^2}$$

Where:

- x_i = Feature of the new data point
- y_i = Feature of a training data point
- d = Distance

2. Classification Rule

Choose the K nearest neighbors.

Example (K = 5):

A
A
B
A
B



Votes:

- A = 3
- B = 2

Prediction:

Class A



3. Regression Rule

Take the average of the K nearest neighbors.

Example:

Nearest house prices:

200
210
190



Prediction:

$$\frac{200 + 210 + 190}{3} = 200$$

Other Distance Formulas

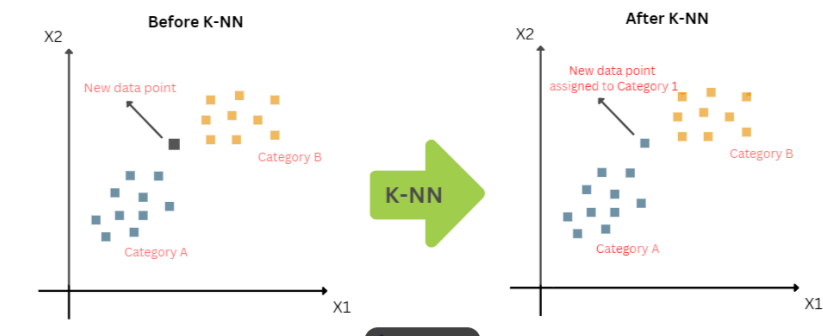
Manhattan Distance

$$d = \sum_{i=1}^n |x_i - y_i|$$

Minkowski Distance

$$d = \left(\sum_{i=1}^n |x_i - y_i|^p \right)^{1/p}$$

- $p = 1 \rightarrow$ Manhattan Distance
- $p = 2 \rightarrow$ Euclidean Distance



 Python



 Run

```
from sklearn.neighbors import KNeighborsClassifier

# Create the model
model = KNeighborsClassifier(n_neighbors=5)

# Train the model
model.fit(X_train, y_train)

# Predict
y_pred = model.predict(X_test)
```

 Python



 Run

```
from sklearn.neighbors import KNeighborsRegressor

model = KNeighborsRegressor(n_neighbors=5)

model.fit(X_train, y_train)

y_pred = model.predict(X_test)
```

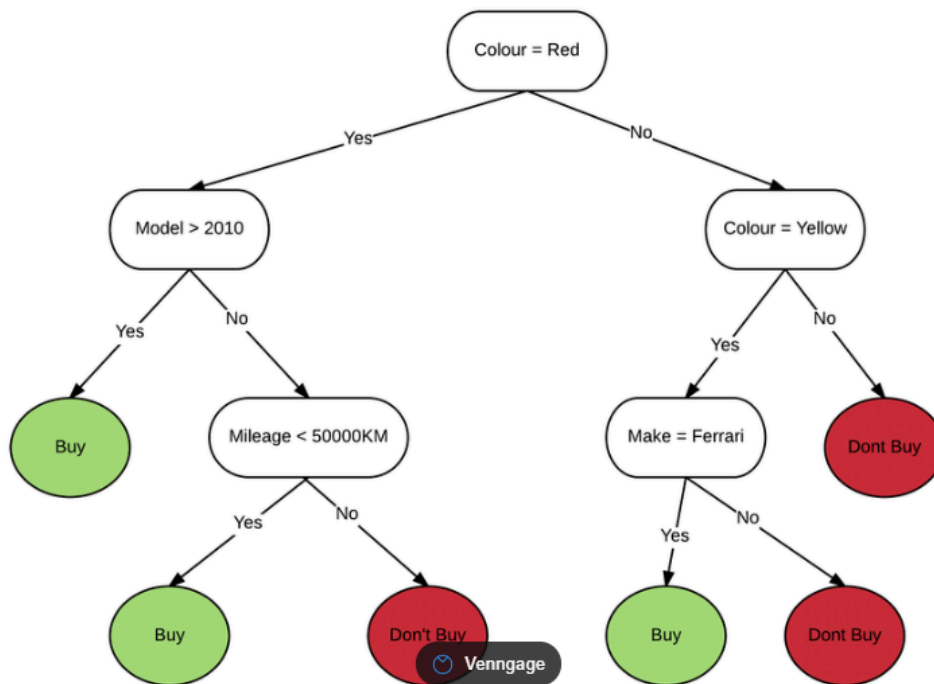
 Python



 Run

```
model = KNeighborsClassifier(
    n_neighbors=5,      # Number of neighbors
    metric="euclidean", # Distance metric
    weights="uniform"   # All neighbors have equal weight
)
```

4. Decision Tree



Python

```
from sklearn.tree import DecisionTreeClassifier

model = DecisionTreeClassifier(
    criterion="gini",      # "gini" or "entropy"
    max_depth=5,          # Maximum depth of the tree
    min_samples_split=2,   # Minimum samples required to split a node
    min_samples_leaf=1,    # Minimum samples required at a leaf node
    max_features=None,     # Number of features to consider for each split
    random_state=42        # For reproducibility
)

model.fit(X_train, y_train)

y_pred = model.predict(X_test)
```

Gini Impurity

$$G = 1 - \sum_{i=1}^n p_i^2$$

Where:

- p_i = Probability of class i

Example:

60% Yes

40% No

$$\begin{aligned} G &= 1 - (0.6^2 + 0.4^2) \\ &= 1 - (0.36 + 0.16) \\ &= 0.48 \end{aligned}$$

Lower Gini = Better split.

Entropy

$$Entropy = - \sum p_i \log_2(p_i)$$

Lower entropy means the node is more pure.

Gini tells us how mixed the data is

🍏🍏🍏🍏 Gini is low.

🍏🍊🍏🍊 Gini is high.

Entropy tells us how confused we are when guessing the class.

🍏🍏🍏🍏 There is no confusion, so Entropy is low.

🍏🍊🍏🍊 Exactly! We are confused. So Entropy is high.

Disadvantages:

- Can overfit

5. Random Forest

```
Python

from sklearn.ensemble import RandomForestClassifier

model = RandomForestClassifier(
    n_estimators=100,      # Number of trees
    criterion="gini",      # or "entropy"
    max_depth=10,         # Maximum tree depth
    min_samples_split=2,
    min_samples_leaf=1,
    max_features="sqrt",  # Features considered at each split
    random_state=42
)

model.fit(X_train, y_train)

y_pred = model.predict(X_test)
```

Random Forest is a bagging ensemble algorithm that trains multiple Decision Trees on random samples of the data and combines their predictions using majority voting (classification) or averaging (regression).

6. Support Vector Machine (SVM)

Support Vector Machine (SVM)

What is SVM?

Support Vector Machine (SVM) is a supervised machine learning algorithm used mainly for classification. It finds the best decision boundary (hyperplane) that separates different classes with the maximum margin.

Formula

Decision Boundary:

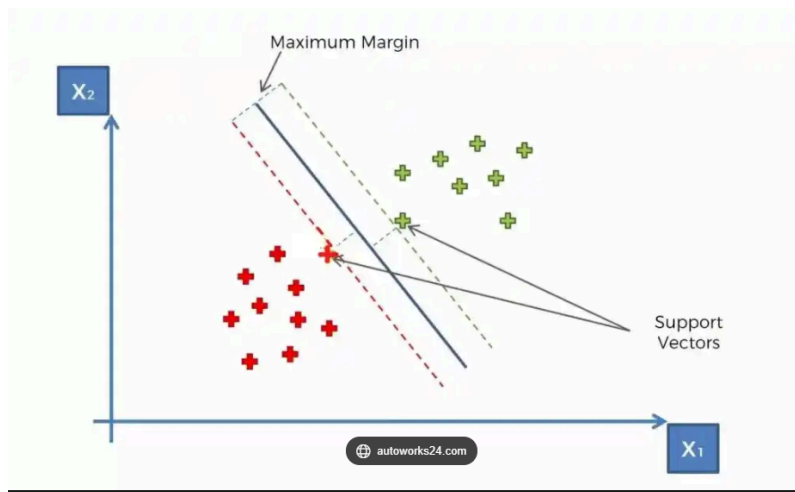
$$w^T x + b = 0$$

Prediction:

$$\text{sign}(w^T x + b)$$

- Positive → Class 1
- Negative → Class 0





Python

```
from sklearn.svm import SVC
model = SVC(kernel="linear", C=1.0, random_state=42)
model.fit(X_train, y_train)
y_pred = model.predict(X_test)
```